

MEMORIA DEL PROYECTO: TRAINBIKE

Ciclo Formativo: Desarrollo de Aplicaciones Web (DAW) - 2º Curso

Autores: Sergio Familiar Hidalgo y Guillermo Lavado

Tecnologías Principales: Laravel (Backend) y Vanilla JavaScript (Frontend)

Fecha: Febrero 2026

1. Arquitectura Base y Desarrollo del Frontend (Cliente)

Para la construcción de la plataforma **TrainBike**, se ha establecido desde el inicio una clara separación de responsabilidades entre el cliente (Frontend) y el servidor (Backend), garantizando así un sistema escalable, seguro y de alto rendimiento.

1.1. Fase 0: Arquitectura Base e Inicialización del Frontend

Para la construcción del cliente de la plataforma, se tomó la decisión arquitectónica de implementar un modelo **Single Page Application (SPA)** apoyado enteramente en Vanilla JavaScript (ES6) y la API del DOM.

El objetivo prioritario desde el inicio ha sido garantizar la máxima seguridad y rendimiento. Por ello, se ha descartado completamente la inserción de cadenas de texto HTML mediante propiedades como `innerHTML` o métodos similares, previniendo así de raíz cualquier tipo de vulnerabilidad Cross-Site Scripting (XSS).

La estructura inicial se basa en los siguientes pilares:

- **Contenedor Único:** Un archivo `index.html` que actúa como contenedor principal (identificado como `#app-container`).
- **Centralización del Estado:** La gestión de estados y navegación se ha centralizado en un archivo `main.js` modular.
- **Inicialización Asíncrona:** Se implementa una función inicializadora (`initApp`) que evalúa de forma asíncrona la existencia de un token de autenticación en el `localStorage`. Esto garantiza que la estructura de la aplicación y la interfaz se construyan dinámicamente.
- **Estilos y UI:** El uso del framework CSS Bootstrap 5 se ha integrado vía CDN para agilizar el maquetado *responsive* sin sobrecargar la lógica de componentes en esta fase inicial.

1.2. Fase 1: Maquetación Segura de la Autenticación mediante <template>

Durante la implementación de los formularios de Registro de usuario y Login, el reto principal

consistía en generar interfaces dinámicas sin comprometer la seguridad. En lugar de construir grandes bloques de HTML concatenando cadenas de texto en JavaScript, se optó por utilizar la etiqueta `<template>` nativa de HTML5.

Esta decisión técnica permitió maquetar visualmente los formularios de autenticación con Bootstrap 5 en el documento principal, pero manteniéndolos inertes (fuera del flujo de renderizado del navegador). El proceso seguido es el siguiente:

1. **Clonación en Memoria:** Mediante el método `content.cloneNode(true)` de la API del DOM, JavaScript clona estas estructuras en memoria.
2. **Asignación de Eventos:** Esta técnica aporta la gran ventaja de permitir capturar las referencias a los nodos (como los formularios o los enlaces de navegación cruzada) y asignarles los `EventListeners` ("click" y "submit") de forma programática.
3. **Inyección Segura:** Una vez preparado el nodo en memoria, se inyecta de forma segura en el contenedor principal usando `appendChild()`.

De esta forma, se cumple rigurosamente con la política de **"cero inyección HTML dinámica"**, protegiendo la plataforma contra vulnerabilidades estructurales mientras se ofrece una navegación SPA instantánea y fluida para el usuario final.

1.3. Fase 2: Consumo Asíncrono de la API (Registro y Login)

Una vez diseñada la estructura de vistas seguras, el siguiente paso fue implementar la comunicación real entre el cliente y el servidor. Cumpliendo con el objetivo general de consumir datos de forma asíncrona, se descartó el envío clásico de formularios HTML que provocan recargas de página.

En su lugar, se utilizó la API nativa `fetch` de JavaScript combinada con el paradigma `async/await` para lograr un flujo de código secuencial más claro y mantenible. En la vista de registro, el cliente captura los valores de los campos (nombre, apellidos, peso, altura, email, contraseña y fecha de nacimiento) extrayendo directamente sus propiedades `.value` del DOM. Con estos datos se construye un objeto literal que posteriormente se serializa mediante `JSON.stringify()`.

Para la autenticación, las peticiones HTTP a los *endpoints* `POST /register` y `POST /login` fueron configuradas con las cabeceras `'Content-Type': 'application/json'` y `'Accept': 'application/json'`. Esto asegura que Laravel responda exclusivamente mediante su arquitectura de API REST.

Una vez que el servidor valida las credenciales y responde con un código HTTP de éxito (2xx), el token de acceso (tipo *Bearer*) se extrae de la respuesta y se almacena en el `localStorage` del navegador. Este almacenamiento persistente permite mantener el estado de la sesión activa en el cliente SPA y sirve como base para inyectarlo en las cabeceras de autorización de las futuras llamadas a las rutas privadas del backend.

1.4. Fase 3: Gestión de Estado, Interceptor Fetch y Cierre de Sesión

Una vez resuelto el acceso, el proyecto exigía que todas las rutas privadas de la API requirieran

autenticación. Para cumplir con este requisito de forma escalable y mantener el principio DRY (*Don't Repeat Yourself*), se desarrolló una función envoltorio (*wrapper*) denominada `fetchAPI`.

Esta función actúa como un interceptor centralizado en el cliente. Antes de lanzar cualquier petición HTTP, verifica la existencia del token de sesión en el `localStorage`. Si el token está presente, inyecta automáticamente la cabecera `Authorization: Bearer <token>`. Además, evalúa la respuesta del servidor de forma global: si detecta un código HTTP 401 Unauthorized en rutas protegidas (indicando que el token ha expirado o es inválido), fuerza automáticamente el cierre de sesión, protegiendo a la aplicación de estados inconsistentes.

Paralelamente, se implementó el cierre de sesión explícito (ruta `/logout`). Al activar esta acción, el cliente realiza una llamada asíncrona a la API para invalidar el token de forma segura en el servidor. Mediante el uso del bloque `finally` de la promesa, se asegura en todo caso la limpieza de los datos en el cliente (`localStorage.removeItem()`), invocando posteriormente de nuevo la función inicializadora para destruir la interfaz protegida y mostrar la vista de acceso. Todo ello logrando una experiencia fluida e ininterrumpida sin recargas de página.

1.5. Fase 4: Manejo Global de Errores y Feedback Visual (UI/UX)

Uno de los requisitos funcionales del frontend consistía en lograr una interfaz clara y usable, gestionando adecuadamente los errores devueltos por el servidor. Para reemplazar las interrupciones bloqueantes (`alert()`) y ofrecer un *feedback* visual de calidad, se desarrolló un sistema de notificaciones global utilizando el componente *Toasts* de Bootstrap 5.

A nivel de implementación, el mayor desafío continuaba siendo el respeto a la política de seguridad estricta frente al XSS. En lugar de generar el código HTML del *Toast* como una cadena de texto, se implementó la función `showToast()`. Esta utilidad dinámica construye nodo a nodo el componente usando la API del DOM (`document.createElement()`, `className`, `setAttribute()` y `textContent`).

Una vez ensamblado en memoria, el nodo se inyecta en un contenedor fijo (`#toast-container`), se instancia mediante el objeto `bootstrap.Toast`, y se programa un `EventListener` para escuchar el evento `hidden.bs.toast`. Este evento es clave, ya que se encarga de destruir el nodo (`.remove()`) una vez que la animación de ocultación ha finalizado, evitando así la saturación y fugas de memoria en el árbol DOM.

Esta solución se integró con la función interceptora (`fetchAPI`) para capturar todas las respuestas del servidor (como credenciales incorrectas, errores de validación o tokens expirados), mostrando alertas semánticas coloreadas (*success*, *danger*, *warning*) que mejoran notablemente la experiencia de usuario.

1.6. Fase 5: Estructuración y Maquetación del Módulo de Bloques (Listado)

Iniciando el desarrollo de los requisitos funcionales del sistema (operaciones CRUD), el primer módulo abordado fue el de "Bloques de entrenamiento". Para la vista de listado, se implementó una interfaz limpia y accesible utilizando las tablas *responsive* y el sistema de tarjetas (*Cards*) de Bootstrap 5.

A nivel de código, se diseñó un nuevo `<template id="tpl-bloques-list">` que define el esqueleto estático de la tabla, estableciendo las cabeceras principales: Nombre, Tipo, Duración, Zonas de intensidad y Acciones.

En el archivo `main.js`, se sustituyó la navegación temporal por un sistema de enrutamiento básico (`router()`) gobernado por una estructura `switch`. Al interceptar el clic en la sección de "Bloques", el sistema limpia el contenedor principal y clona el nodo del *template* en memoria.

Se dejó preparado intencionadamente un elemento `<tbody>` vacío con una fila temporal indicadora de carga ("Cargando..."). Esta estructura queda lista para ser reemplazada en la siguiente fase de desarrollo mediante la inyección dinámica de nodos de texto, una vez se resuelva con éxito la petición asíncrona al backend.

1.7. Fase 6: Consumo y Renderizado Dinámico de Bloques (DOM Scripting)

Una vez preparado el contenedor y el diseño de la tabla de bloques, el siguiente paso lógico consistió en hidratar la interfaz con datos reales provenientes de la base de datos.

Para lograrlo, se implementó la función asíncrona `fetchBloques()`, que utiliza el interceptor desarrollado en fases previas para solicitar el listado completo de bloques a la ruta protegida `/api/bloque` (mediante una petición GET). El punto técnico de mayor interés reside en la función encargada del renderizado de filas: `renderBloquesRows()`.

Esta función recibe el Array de objetos JSON devuelto por Laravel y genera la estructura tabular elemento por elemento utilizando la API nativa de JavaScript. Iterando sobre los datos, se instancian nodos mediante `document.createElement('tr')` y `document.createElement('td')`, asignando el contenido de variables como la duración estimada o las zonas de intensidad (potencia y pulso) de forma segura utilizando la propiedad `.textContent`.

Este patrón no solo previene de forma efectiva los ataques de *Cross-Site Scripting* (XSS), sino que permite adjuntar eventos directos a los botones de "Ver" y "Eliminar" de cada fila durante la propia construcción del DOM. Esta estrategia optimiza el ciclo de vida de los eventos y evita depender de una delegación masiva en el contenedor padre.

1.8. Fase 7: Maquetación y Validación Frontend (Formulario de Bloques)

Avanzando en el desarrollo del CRUD para la entidad `bloque_entrenamiento`, se diseñó la

interfaz para la inserción de nuevos registros. Basándose rigurosamente en el modelo entidad-relación del proyecto, el formulario incluye campos para el nombre, descripción, tipo (estructurado mediante un elemento `<select>`), duración estimada y métricas de rendimiento detalladas como los porcentajes de FTP (potencia) y pulso.

Desde el punto de vista arquitectónico y de experiencia de usuario, se delegó la primera capa de validación en las APIs nativas de HTML5, aprovechando atributos semánticos como `required`, `type="number"`, `min` y `max`. Esta decisión asegura que los valores introducidos (como los porcentajes de esfuerzo) no superen rangos lógicos, validando la información en el cliente antes de generar tráfico de red innecesario hacia el servidor.

En la capa de lógica JavaScript, se inyectó el nuevo `<template>` del formulario en el DOM y se interceptó el evento `submit` mediante la instrucción `e.preventDefault()`. Esta técnica detiene el envío síncrono por defecto, preparando el controlador de la vista para serializar los datos y ejecutar la petición POST asíncrona en la siguiente fase evolutiva del desarrollo. Todo ello manteniendo la navegación instantánea y sin pérdida de estado típica de las aplicaciones SPA.

1.9. Fase 8: Persistencia Asíncrona (POST) y Reactividad de la UI (Bloques)

Una vez diseñada la vista del formulario, se procedió a desarrollar el controlador JavaScript `handleBloqueSubmit()` para gestionar la captura y persistencia asíncrona de la entidad `bloque_entrenamiento`.

La arquitectura implementada extrae de forma limpia los valores del DOM, transformando dinámicamente las cadenas vacías de los *inputs* numéricos (como los porcentajes de potencia y pulso) en valores null mediante una función auxiliar `getIntOrNull()`. Esto garantiza que el *payload* JSON enviado al servidor mantenga una estricta consistencia con los tipos de datos requeridos por la base de datos de Laravel, previniendo excepciones de casteo numérico.

La petición se ejecuta a través de la ruta protegida POST `/api/bloque/crear` utilizando la capa de abstracción `fetchAPI` desarrollada previamente, asegurando la inyección automática del token de sesión.

Tras obtener una respuesta satisfactoria del servidor (código HTTP 200/201), el cliente no requiere forzar una recarga de página: el *script* notifica el éxito mediante un *Toast* e invoca inmediatamente al método de ruteo `renderBloquesList()`. Este patrón arquitectónico reactivo destruye el nodo del formulario, reconstruye la vista tabular y dispara automáticamente una nueva petición GET en segundo plano. Esto garantiza que la tabla refleje el estado real y actualizado de la base de datos sin fricción visual para el usuario.

1.10. Fase 9: Detalles y Eliminación de Entidades (Lectura y Borrado)

Para completar el ciclo de gestión CRUD de la entidad `bloque_entrenamiento`, se

implementaron las funcionalidades de lectura individual y borrado.

En primer lugar, se abordó la visualización detallada (GET /api/bloque/{id}). Se diseñó un nuevo *template* HTML (#tpl-bloque-detalle) estructurado con clases de utilidad de Bootstrap. En la capa de JavaScript, la función `fetchBloqueDetalle(id)` realiza la petición asíncrona inyectando el ID del bloque en la URL. Tras resolver la promesa, los datos se mapean de forma segura a la interfaz utilizando exclusivamente la propiedad `.textContent`. Esto asegura que el contenido (como las descripciones largas o los comentarios de texto libre insertados por el usuario) nunca sea interpretado como HTML, neutralizando posibles vectores de ataque XSS.

En segundo lugar, se implementó el proceso de destrucción de registros. Según las especificaciones de la API, esto requiere invocar el *endpoint* DELETE /api/bloque/{id}/eliminar. Para ello, se añadió la función `deleteBloque(id, nombre)` que, previa interceptación mediante un diálogo `confirm()` nativo para prevenir pérdidas de datos accidentales, lanza la petición HTTP DELETE. Si el servidor confirma la operación (código HTTP 200), el cliente notifica el éxito al usuario mediante el sistema global de notificaciones e invoca nuevamente a `renderBloquesList()`. Esta acción sincroniza reactivamente el DOM con el nuevo estado de la base de datos, culminando así un flujo CRUD robusto y completamente asíncrono.

1.11. Fase 10: Listado y Creación Asíncrona de Planes de Entrenamiento

Tras completar y consolidar la arquitectura para los Bloques de entrenamiento, se replicó este patrón de diseño comprobado para el módulo de Planes de Entrenamiento. Para ello, se añadieron dos nuevos elementos `<template>` destinados a gestionar la vista de la tabla de registros y el formulario de inserción.

El formulario de creación captura las fechas planificadas, la descripción y un interruptor booleano (mapeado de forma binaria para su persistencia en la base de datos relacional) que indica si el plan se encuentra "Activo". Al procesar el evento `submit`, el cliente JavaScript captura los datos utilizando puramente la API del DOM, construye la estructura de la petición asíncrona hacia el *endpoint* POST /api/plan/crear, y gestiona la visualización del proceso.

Durante el desarrollo de esta fase, se tuvo especial consideración con la dependencia estructural del modelo de datos (`id_ciclista`), asegurando que el *payload* enviado cumpliera estrictamente con todas las reglas de validación establecidas en el `PlanController` del backend. Al igual que en el módulo de bloques, la interfaz se refresca reactivamente mostrando el nuevo registro de la base de datos mediante la API nativa y sin requerir intervención ni recargas manuales por parte del usuario.

1.12. Fase 11: Reutilización de Componentes, Edición (PUT) y Borrado (DELETE)

Para finalizar el módulo de Planes de Entrenamiento, se procedió a implementar las

funcionalidades de modificación (PUT /api/plan/{id}) y eliminación (DELETE /api/plan/{id}) indicadas en las especificaciones de la API.

Para optimizar el código y adherirse estrictamente al principio DRY (*Don't Repeat Yourself*), se evitó la duplicación de código en las vistas HTML. Se modificó la función `renderPlanForm(plan = null)` para que aceptara opcionalmente un objeto completo. Cuando el usuario acciona el botón "Editar", la función inyecta los valores actuales del plan en los *inputs* del DOM del `<template>` clonado, alterando dinámicamente el título del formulario y el flujo de la petición.

El controlador del evento `submit` (`handlePlanSubmit`) se actualizó para ser polimórfico: evalúa si existe un identificador (`planId`) en su contexto de ejecución. En caso afirmativo, formatea la petición HTTP para utilizar el método PUT hacia la ruta `/api/plan/{id}`; en caso negativo, utiliza POST hacia `/api/plan/crear`.

Finalmente, la función `deletePlan` maneja el borrado asíncrono, incluyendo una alerta de prevención (`confirm()`) nativa previa a la ejecución de la petición. Todo este flujo de trabajo se realiza alterando y destruyendo los nodos del DOM en memoria antes de reinyectarlos en el contenedor principal, lo que asegura el rendimiento óptimo característico de una SPA y erradica por completo cualquier vulnerabilidad de inyección XSS.

1.13. Fase 12: Arquitectura Frontend para el Scroll Infinito (Sesiones)

El requisito funcional de mayor complejidad técnica estipulado en el proyecto ha sido la implementación de una carga asíncrona avanzada mediante *Scroll Infinito* para el listado de sesiones de entrenamiento. La directriz principal del proyecto prohíbe explícitamente el uso de una paginación clásica con recarga de página.

Para acometer este desafío, esta fase se dedicó al planteamiento arquitectónico de la vista. Se diseñó un nuevo `<template id="tpl-sesiones-list">` modificando el patrón de presentación: en lugar de una tabla estricta, se optó por un contenedor de cuadrícula (*Grid*) que alojará tarjetas de resumen (*Cards*). Este diseño favorece enormemente la experiencia de usuario durante un desplazamiento vertical continuo, adaptándose mejor a distintos dispositivos.

Técnicamente, el componente más crítico introducido en el DOM es el elemento `#scroll-sentinel` (un nodo centinela). La estrategia moderna de desarrollo Frontend aplicada descarta la vinculación directa de eventos de escucha (*event listeners*) sobre la acción de *scroll* de la ventana, ya que esta práctica tradicional degrada severamente el rendimiento y bloquea el hilo principal de ejecución del navegador. En su lugar, este "centinela" actúa como un objetivo pasivo que será vigilado por la API nativa `IntersectionObserver` de JavaScript. Esta herramienta se implementará en la siguiente fase para disparar las peticiones de paginación al servidor de forma extremadamente eficiente y fluida.

1.14. Fase 13: Implementación del Motor de Scroll Infinito (Intersection

Observer)

La lógica de cliente del *scroll infinito* requería una implementación altamente optimizada para no penalizar el rendimiento del renderizado en el navegador. Por este motivo, se descartó la escucha tradicional del evento `window.onscroll` y se optó por la moderna API `IntersectionObserver`.

El desarrollo implicó definir un estado local en el módulo de sesiones, controlado por las variables `sesionesOffset` (puntero de inicio) y `sesionesLimit` (tamaño de página). En la inicialización de la vista, se instancia el observador (`initIntersectionObserver`), configurándolo para vigilar el nodo `#scroll-sentinel` con un *threshold* de 0.1 (disparo al 10% de visibilidad).

El *callback* de intersección está protegido por *guards* lógicos (`!isFetchingSesiones` y `hasMoreSesiones`) para prevenir múltiples peticiones de red simultáneas (condición de carrera) en caso de que el usuario haga *scroll* de forma errática o muy rápida.

Además, se implementó una rutina de recolección de basura (*garbage collection*) en la función `clearAppContainer()`. Esto asegura que el método `.disconnect()` del observador se invoque sistemáticamente al desmontar la vista, evitando así fugas de memoria (*memory leaks*) y efectos secundarios no deseados al navegar hacia otros módulos de la SPA.

1.15. Fase 14: Consumo Paginado de la API REST (Requisito Clave)

En esta iteración se abordó la integración real del motor de *scroll infinito* con el controlador del backend, satisfaciendo el requisito clave de consumo asíncrono avanzado establecido en el proyecto.

A nivel lógico, el cliente construye dinámicamente el *endpoint* inyectando los parámetros `offset` y `limit` (por ejemplo, `GET /api/sesion?offset=0&limit=10`) estipulados en la documentación técnica. La promesa devuelta por la utilidad `fetchAPI` se procesa y sus resultados se envían a la función `renderSesionesCards()`. Esta función genera el árbol DOM de las tarjetas (basadas en el componente *Card* de Bootstrap) de forma completamente programática y segura (sin utilizar `innerHTML`), enlazando los datos devueltos (como el nombre, la fecha o el estado booleano de completada) a las propiedades `.textContent` y `.className` de los nodos instanciados.

Para proteger la experiencia de usuario y optimizar el rendimiento, se implementó una inferencia matemática vital: si el servidor devuelve un *array* cuya longitud (`length`) es estrictamente menor que el límite solicitado (`sesionesLimit`), el frontend deduce que ha alcanzado el final absoluto de los registros en la base de datos. Bajo esta condición de parada, la variable de estado `hasMoreSesiones` se establece en `false`, el nodo centinela se oculta y se muestra un mensaje informativo en pantalla, liberando definitivamente a la API `IntersectionObserver` de tareas de vigilancia y evitando peticiones estériles a la red.

1.16. Fase 15: Operaciones de Escritura y Relaciones Foráneas

(Sesiones)

Para concluir el módulo base de Sesiones de Entrenamiento, se desarrollaron los flujos de creación (POST /api/sesion/crear) y borrado (DELETE /api/sesion/{id}).

El mayor reto técnico de esta fase radicaba en mantener la integridad referencial del modelo de datos: una sesión pertenece obligatoriamente a un plan de entrenamiento (id_plan). Para abordar esta restricción desde el cliente sin comprometer la usabilidad, se diseñó la función asíncrona fetchPlanesForSelect(). Al cargar la vista del formulario, el frontend dispara de forma concurrente una petición GET a la ruta /api/plan. Tras procesar la respuesta, inyecta programáticamente nodos <option> en el elemento <select> del formulario. Esto permite al usuario relacionar entidades de forma completamente transparente y asegura que el *payload* enviado al servidor contenga claves foráneas válidas.

En cuanto a la eliminación de registros, se conectó el evento de los botones generados dinámicamente en las tarjetas (implementadas en la Fase 14) hacia la función deleteSesion(). Dado que el contenedor emplea un sistema de *scroll infinito* con un estado interno mutado (controlado por las variables de offset), se tomó una decisión arquitectónica crucial: tras completarse el borrado (DELETE) con éxito en el servidor, se fuerza la reinicialización completa de la vista invocando a renderSesionesList(). Esta acción purga la caché visual y resetea los contadores del *scroll*, garantizando que el usuario visualice un estado del DOM 100% sincronizado con la paginación real de la base de datos y previniendo desajustes en la interfaz tras la eliminación.

1.17. Fase 16: Interfaz de Relaciones Complejas (Sesión-Plan)

El proyecto exigía la gestión y visualización de relaciones anidadas, específicamente la asignación de sesiones a los planes de entrenamiento. Aprovechando el *endpoint* GET /api/sesionPlan desarrollado en el backend (el cual utiliza el método with(['sesiones']) de Eloquent ORM para realizar *Eager Loading* y prevenir el clásico problema de rendimiento $N+1$), se procedió a diseñar la capa de presentación correspondiente.

Para representar esta relación jerárquica (1:N desde la perspectiva del plan), se implementó el componente *Accordion* (Acordeón) de Bootstrap. La función renderSesionPlanAccordion() procesa la matriz JSON devuelta por el servidor iterando en dos niveles:

1. Primero, itera sobre la colección de planes para generar las cabeceras colapsables principales (accordion-header).
2. A continuación, realiza una iteración anidada sobre la propiedad vinculada plan.sesiones para generar una sublista (list-group) dentro de cada cuerpo del acordeón (accordion-body).

Toda esta estructura compleja, que incluye múltiples atributos HTML y clases CSS condicionadas dinámicamente (como collapse show frente a collapse), se ensambló rigurosamente mediante la API del DOM (document.createElement). Esta decisión

arquitectónica garantiza la máxima fluidez de la SPA al tiempo que mantiene una barrera de seguridad infranqueable contra posibles vulnerabilidades de inyección XSS.

1.18. Fase 17: Generación de Formularios Dinámicos y Payloads Anidados (Sesión-Plan)

Para aprovechar la robustez del método `crearSesionPlan` desarrollado en el backend (capaz de persistir relaciones complejas en una sola transacción mediante un *array* anidado de datos), se diseñó un formulario dinámico e interactivo en el cliente.

A nivel de arquitectura del DOM, se implementó el patrón de Plantillas Auxiliares (*Auxiliary Templates*). Se definió un `<template id="tpl-sesion-row">` que encapsula la estructura HTML de una única sesión (conteniendo sus *inputs* y un botón de eliminación individual). Mediante la función `addSesionRow()`, el usuario puede clonar e inyectar este fragmento en la vista de forma ilimitada usando el método `cloneNode(true)`.

En el momento de realizar el envío del formulario (*submit*), la función `handleSesionPlanSubmit()` realiza un barrido del DOM capturando todas las instancias de la clase `.session-row` mediante `querySelectorAll`. Iterando sobre esta lista de nodos, la función extrae los valores individuales de cada fila para construir lógicamente el arreglo `sesionesArray`.

Posteriormente, este arreglo dinámico se inyecta como una propiedad dentro del objeto literal del Plan, generando así un *payload* JSON estructurado de forma jerárquica. Este flujo de trabajo asegura que los datos enviados cumplan estrictamente con las reglas de validación avanzadas (por ejemplo, validaciones de arrays anidados como `'sesiones.*.fecha'`) definidas en el controlador de Laravel.

1.19. Fase 18: Gestión de la Tabla Pivote (Sesión-Bloque)

Para cumplir íntegramente con los requisitos de la API referentes a la composición interna de los entrenamientos, se desarrolló una vista dedicada exclusivamente a la gestión de la tabla pivote `sesion_bloque`.

A nivel de interfaz de usuario (UI), la vista implementa un patrón Maestro-Detalle bidireccional: un formulario de vinculación situado a la izquierda y una tabla de relaciones activas a la derecha. Para alimentar los selectores (`<select>`) del formulario de forma eficiente, se utilizó el método `Promise.all()`. Esta decisión técnica permite ejecutar las peticiones `GET /api/sesion` y `GET /api/bloque` en paralelo (conurrencia), reduciendo drásticamente el tiempo de carga del catálogo de opciones y mejorando notablemente la percepción de rendimiento por parte del usuario.

El flujo de escritura gestiona la creación de la nueva relación de forma asíncrona mediante el *endpoint* `POST /api/sesionbloque`. Por otro lado, la funcionalidad crítica de eliminación se resolvió enlazando dinámicamente el ID de la relación (la clave primaria de la tabla pivote

autogenerada por Laravel) al botón de "Desvincular" presente en la tabla. Al accionar este evento, el cliente invoca el *endpoint* específico DELETE /api/sesionbloque/{id}. Esta operación garantiza que la relación polimórfica o de muchos a muchos (N:M) se rompa de forma segura y aislada, sin afectar ni a la sesión original ni al bloque de entrenamiento subyacente.

1.20. Fase 19: Registro de Métricas de Rendimiento (Resultados)

En esta fase del desarrollo se implementó la lógica para capturar el rendimiento empírico del deportista, permitiendo cerrar el ciclo del entrenamiento. Para ello, se diseñó un formulario dinámico que permite vincular unívocamente una sesión previamente planificada con sus métricas reales de ejecución.

A nivel de diseño de interfaz, destaca la incorporación de un selector de rango iterativo (mediante un input de tipo range) para capturar el RPE (*Rating of Perceived Exertion* o Percepción Subjetiva del Esfuerzo). Esta decisión facilita una entrada de datos mucho más visual e intuitiva para el usuario en dispositivos de cualquier formato.

En la capa de negocio del cliente, el controlador JavaScript se encarga de serializar todas estas métricas en un objeto de datos. Al invocar la petición asíncrona hacia el *endpoint* POST /api/resultado/crear, el *script* ejecuta un proceso de pre-formateo, asegurando explícitamente que los valores críticos, como la potencia media y el pulso cardíaco, sean parseados y tratados como números enteros de forma estricta. Esta precaución en el lado del cliente garantiza la integridad relacional con el esquema definido en la base de datos de Laravel, previniendo excepciones y errores de validación de tipos en el backend.

2. Decisiones Técnicas y Proceso de Desarrollo (Backend)

Para dar cumplimiento estricto a los requisitos del proyecto, que exigen una separación clara entre el frontend y el backend mediante una API REST, el backend en Laravel se ha configurado exclusivamente como proveedor de datos y gestor de lógica de negocio.

2.1. Arquitectura Cliente-Servidor y Enrutamiento

La implementación del cliente como una SPA implicó una reestructuración fundamental en el enrutamiento por defecto de Laravel:

- **Limpieza de web.php y Vistas Tradicionales:** Se eliminaron las rutas duplicadas y el comando Auth::routes(); del archivo web.php, así como las carpetas /auth y /layouts generadas por *laravel/ui*. Al ser una SPA asíncrona, la aplicación web no debe recargar la página ni depender de vistas de Blade para la navegación o el login. El archivo web.php quedó configurado exclusivamente con una ruta raíz (/) y un *fallback* que devuelven un único lienzo: app.blade.php.
- **Centralización en api.php:** Toda la lógica de negocio y comunicación con la base de

datos se trasladó a routes/api.php, garantizando respuestas exclusivamente en formato JSON y manteniendo el backend sin estado (*stateless*).

2.2. Autenticación y Seguridad (Laravel Sanctum)

Se implementó la autenticación mediante tokens de sesión utilizando **Laravel Sanctum**.

Para ello, se desarrolló el controlador AuthController integrando los métodos necesarios:

- **Login y Registro:** Se implementó tanto el login como el método register exigido por los requisitos funcionales.
- **Validación Estricta:** En el registro, se validan estrictamente los datos del ciclista.
- **Seguridad:** Se encripta la contraseña antes de persistirla en la base de datos, retornando un access_token seguro para ser almacenado y gestionado por el cliente.

2.3. Refactorización y Buenas Prácticas (Clean Code)

Durante el desarrollo y revisión del código de los controladores, se aplicaron diversas mejoras de calidad y principios de *Clean Code*:

- **Paginación para Scroll Infinito:** En el SesionController, el *endpoint* para listar sesiones requería soportar los parámetros offset y limit. Se refactorizó la captura de estos parámetros, sustituyendo el uso de variables superglobales de PHP (\$_GET) por el objeto estandarizado e inyectado Illuminate\Http\Request (\$request->query()). Esto aumenta significativamente la seguridad y testeabilidad del código.
- **Integridad Relacional en Planes de Entrenamiento:** Se desarrolló una lógica avanzada en el PlanController para manejar la relación N:M entre planes y sesiones.
 - Se implementó el método crearSesionPlan validando arrays anidados de sesiones.
 - Se creó un método deleteSesionPlan que realiza un borrado en cascada manual y seguro: primero desvinculando la tabla pivote de bloques (detach()), luego eliminando las sesiones asociadas, y finalmente el plan, previniendo así excepciones de restricciones de claves foráneas.

2.4. Resolución de Problemas: Arquitectura de Base de Datos y Registro Doble

Durante la implementación del registro de usuarios y su integración con el frontend, surgió un reto técnico importante reflejado en el error de base de datos: SQLSTATE[42S22]: Column not found: 1054 Unknown column 'email' in 'where clause'.

Al analizar la traza, se detectó que el error provenía de la regla de validación del controlador (unique:ciclistas). Esto se debió a una decisión arquitectónica previa: el modelo de datos físico diseñado separa conceptualmente las responsabilidades en dos tablas distintas, divergiendo de un modelo monolítico para favorecer la **normalización de la base de datos**:

1. **Tabla users:** Estructura estandarizada (id, name, email, password), encargada puramente de la autenticación y compatible de forma nativa con Laravel Sanctum.
2. **Tabla ciclistas:** Estructura extendida (id_user, apellido, fecha_nacimiento, peso_base,

altura_base), ligada mediante clave foránea a la tabla de usuarios, encargada de almacenar las métricas físicas y el perfil deportivo del atleta.

Solución Técnica Implementada:

Para mantener la interfaz de usuario de la SPA intacta (el formulario del cliente sigue enviando un único objeto JSON unificado con todos los datos), se modificó la lógica interna del método register en el AuthController del backend:

- En primer lugar, se corrigió la regla de validación para que verificara la unicidad del correo electrónico en la tabla correcta ('email' => 'unique:users').
- En segundo lugar, se implementó un **patrón de inserción secuencial**: el controlador ahora crea primero la entidad User, extrae su ID recién autogenerado y, posteriormente, crea la entidad Ciclista inyectando dicho ID como clave foránea (id_user).

De este modo, se consiguió mantener la base de datos perfectamente normalizada, aprovechar el sistema de autenticación robusto de Laravel, y cumplir rigurosamente con los requisitos del frontend sin alterar la experiencia asíncrona del usuario.